

Lab4: 进程管理

南开大学 计算机学院和密码与网络空间安全学院

- 年级: 2023级
- 专业: 信息安全
- 姓名及学号: 于昊汐(2312642)、陈安琪(2312481)、孙丽菲(2312724)
- 日期: 2025年11月20日

一、实验目的

- 理解虚拟内存管理的核心机制,掌握虚拟地址空间的组织逻辑、多级页表的地址转换原理,以及页表项的创建、映射与删除等关键操作。
- 掌握内核线程的核心概念与特性,明确其与用户进程的差异,理解进程控制块(PCB)的结构与作用,熟悉内核线程的创建、初始化流程。
- 深入学习进程调度与上下文切换的实现原理,掌握FIFO调度策略的执行逻辑,理解中断控制、页表切换与寄存器上下文保存/恢复的协同过程。
- 实现从单一执行流内核到多线程并发内核的扩展,搭建基本的虚拟内存环境框架,为后续用户进程、系统调用、缺页异常处理等功能奠定基础。
- 熟练运用ucore OS的内存管理接口和进程管理相关数据结构,提升操作系统核心功能的编码与调试能力。

二、实验内容

实验基于前序物理内存管理和基础页表机制,重点完成虚拟内存管理完善与内核线程并发执行两大核心任务,具体内容如下:

1. 核心练习:进程管理核心功能实现

练习1:分配并初始化进程控制块: 在 `kern/process/proc.c` 的 `alloc_proc` 函数中,完成 `struct proc_struct` (进程控制块PCB)的分配与初始化。需对进程状态、进程ID(pid)、页目录基址、内核栈、上下文、中断帧等关键成员变量进行初始化,为内核线程提供基础管理结构。

练习2:为新创建内核线程分配资源: 完善 `kern/process/proc.c` 的 `do_fork` 函数,实现内核线程的创建流程。核心步骤包括:调用 `alloc_proc` 获取PCB、为新线程分配独立内核栈、复制原进程的上下文与中断帧、将新进程添加到进程链表与哈希表、唤醒新进程并返回其pid,确保新线程具备独立执行的资源条件。

练习3:编写进程切换执行函数: 实现 `proc_run` 函数,完成指定进程到CPU的切换。需先检查进程是否为当前运行进程,通过 `local_intr_save` 和 `local_intr_restore` 宏控制中断开关,切换当前进程标识、更新页表(调用 `lsatp` 函数修改SATP寄存器),并调用 `switch_to` 函数完成进程上下文的切换,保障多线程并发执行。

2. 扩展挑战(Challenge)

分析 `local_intr_save(intr_flag)` 与 `local_intr_restore(intr_flag)` 宏的实现逻辑,理解其关闭和恢复中断的底层原理,明确中断控制在进程切换中的作用。

深入研究sv32、sv39、sv48分页模式的异同,解释 `get_pte` 函数中两段相似代码的设计原因;评估该函数中将页表项查找与分配合并的设计合理性,探讨功能拆分的可行性与优势。

三、实验过程

练习1:分配并初始化一个进程控制块(需要编码)

`alloc_proc` 函数(位于 `kern/process/proc.c` 中)负责分配并返回一个新的 `struct proc_struct` 结构,用于存储新建立的内核线程的管理信息。ucore需要对这个结构进行最基本的初始化,你需要完成这个初始化过程。

请在实验报告中简要说明你的设计实现过程。请回答如下问题:

请说明 `proc_struct` 中 `struct context context` 和 `struct trapframe *tf` 成员变量含义和在本实验中的作用是啥?(提示通过看代码和编程调试可以判断出来)

Ans: `alloc_proc` 函数是ucore操作系统进程管理模块的基础核心函数,其核心功能是分配进程控制块并初始化其所有成员变量,为后续内核线程/进程的创建(如`idleproc`、`initproc`)提供合法的PCB实例。

这里按照注释的提示完成相关初始化,实现如下:

```
// alloc_proc - alloc a proc_struct and init all fields of proc_struct
static struct proc_struct *
alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL)
    {
        proc->state = PROC_UNINIT;           // 初始状态为未初始化
        proc->pid = -1;                       // PID暂设为非法值-1(后续由get_pid分配)
        proc->runs = 0;                       // 进程运行次数初始化为0
        proc->kstack = 0;                     // 内核栈地址暂设为0(后续由setup_kstack
        // 分配)
        proc->need_resched = 0;               // 初始不需要调度(0表示false)
        proc->parent = NULL;                  // 父进程初始为NULL
        proc->mm = NULL;                      // 内核线程无独立内存空间,mm设为NULL
        memset(&proc->context, 0, sizeof(struct context)); // 上下文寄存器清零
        proc->tf = NULL;                      // 中断帧暂设为NULL(后续由copy_thread
        // 构造)
        proc->pgdir = boot_pgdir_pa;         // 共享内核页表的物理地址
        proc->flags = 0;                       // 进程标志位初始为0(无特殊属性)
        memset(proc->name, 0, PROC_NAME_LEN + 1); // 进程名缓冲区清零
        list_init(&proc->list_link);           // 初始化进程链表节点(用于加入全局进程链
        // 表)
        list_init(&proc->hash_link);           // 初始化进程哈希表节点(用于按PID快速查
        // 找)
    }
    return proc;
}
```

先来说明一下这个函数的设计思路:

1. 所有成员必须显式初始化,避免未定义行为(如野指针、随机值导致的调度/内存错误);
2. 默认值需符合ucore进程模型规范(如初始状态为PROC_UNINIT、内核线程共享内核页表);
3. 依赖后续函数完成的初始化(如内核栈、PID、中断帧),暂设为非法值或NULL,明确职责边界。

这是对初始化逻辑的说明:

成员变量	初始化值	设计理由
state	PROC_UNINIT	进程刚分配时未完成任何配置,处于"未初始化"状态
pid	-1	PID需通过get_pid分配唯一合法值(1~MAX_PID),-1表示未分配
kstack	0	内核栈由setup_kstack函数分配物理页并映射,此处暂设为无效地址
need_resched	0	ucore无bool类型,用0表示"不需要调度",1表示"需要调度"
parent	NULL	进程创建时父进程尚未指定(由do_fork后续设置)
mm	NULL	内核线程共享内核地址空间,无需独立内存管理结构
context	全零	上下文寄存器(ra、sp、s0-s11)需在切换前填充,初始清零避免脏数据影响
tf	NULL	中断帧由copy_thread构造在kernel stack顶部,此处暂不分配
pgdir	boot_pgdir_pa	内核线程共享内核页表,直接使用启动阶段初始化的boot_pgdir物理地址
name	全零	进程名由set_proc_name后续设置,初始清零避免残留垃圾字符
list_link/hash_link	链表初始化	进程需加入全局进程链表(proc_list)和PID哈希表(hash_list),提前初始化节点

整体的实现步骤为:

1. 调用 `kmalloc` 分配 `struct proc_struct` 大小的内存;
2. 若内存分配成功,按上述逻辑逐一初始化所有成员变量;
3. 返回初始化后的PCB指针(分配失败返回NULL)。

接下来回答练习1中提出的问题:

`struct context context` 的含义及作用:

- **含义:** 存储线程切换所需的被调用者保存寄存器集合,包括返回地址(ra)、栈指针(sp)、保存寄存器(s0~s11),共14个寄存器。
- **作用:** 实现线程上下文切换的核心载体。当发生调度时, `switch_to` 函数会将当前线程的context保存到你PCB中,同时从目标线程的PCB中恢复context到CPU寄存器,使目标线程从之前的执行现场继续运行。而仅保存线程切换必需的寄存器,不包含所有通用寄存器,切换效率更高。

`struct trapframe *tf` 的含义及作用:

- **含义:** 指向中断帧的指针,中断帧存储了中断/异常发生时的完整进程状态,包括所有通用寄存器(a0~a7、t0~t6、s0~s11等)、状态寄存器(sstatus)、程序计数器(epc)、异常原因寄存器(scause)等。
- **作用:**
 1. 特权级切换时,保存进程的完整执行现场,确保中断处理完成后能恢复原执行流程;

2. 内核线程创建时,由 `copy_thread` 构造中断帧,指定线程的入口函数(epc)、参数(a0/a1)和栈指针(sp),作为线程的初始执行模板;
3. 系统调用时,通过修改中断帧中的寄存器值(如a0)传递返回结果,实现内核态到用户态的数据交互。

练习2:为新创建的内核线程分配资源(需要编码)

创建一个内核线程需要分配和设置好很多资源。`kernel_thread` 函数通过调用 `do_fork` 函数完成具体内核线程的创建工作。`do_kernel` 函数会调用 `alloc_proc` 函数来分配并初始化一个进程控制块,但 `alloc_proc` 只是找到了一小块内存用以记录进程的必要信息,并没有实际分配这些资源。`ucore`一般通过 `do_fork` 实际创建新的内核线程。`do_fork` 的作用是,创建当前内核线程的一个副本,它们的执行上下文、代码、数据都一样,但是存储位置不同。因此,我们实际需要"fork"的东西就是stack和trapframe。在这个过程中,需要给新内核线程分配资源,并且复制原进程的状态。你需要完成在 `kern/process/proc.c` 中的 `do_fork` 函数中的处理过程。它的大致执行步骤包括:

1. 调用 `alloc_proc`,首先获得一块用户信息块;
2. 为进程分配一个内核栈;
3. 复制原进程的内存管理信息到新进程(但内核线程不必做此事);
4. 复制原进程上下文到新进程;
5. 将新进程添加到进程列表;
6. 唤醒新进程;
7. 返回新进程号;

请在实验报告中简要说明你的设计实现过程。请回答如下问题:

请说明`ucore`是否做到给每个新fork的线程一个唯一的id?请说明你的分析和理由。

Ans: `do_fork()` 是uCore中创建新进程/线程的核心函数,它负责完成从PCB分配到新进程变为可运行状态的全部准备工作。

这里按照注释的提示完成每一步的实现,具体如下:

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    int ret = -E_NO_FREE_PROC;
    struct proc_struct *proc;
    if (nr_process >= MAX_PROCESS)
    {
        goto fork_out;
    }
    ret = -E_NO_MEM;

    // 1. 分配并初始化进程控制块
    proc = alloc_proc();
    if (!proc) {
        goto fork_out; // PCB分配失败
    }

    // 2. 分配并初始化内核栈
    if (setup_kstack(proc) != 0) {
        goto bad_fork_cleanup_proc; // 分配内核栈失败,需要释放PCB
    }

    // 3. 根据clone_flags决定是复制还是共享内存管理系统
    if (copy_mm(clone_flags, proc) != 0) {
```

```

        goto bad_fork_cleanup_kstack; // 复制mm失败,需要释放内核栈+PCB
    }

    // 4. 设置进程的中断帧和上下文
    copy_thread(proc, stack, tf);

    // 5. 把设置好的进程加入链表
    proc->pid = get_pid(); // 调用get_pid()分配唯一的pid
    list_add(&proc_list, &proc->list_link); // 插入双向循环链表proc_list
    hash_proc(proc); // 插入哈希表hash_list
    nr_process++; // 全局进程计数+1

    // 6. 将新的进程设为就绪态
    wakeup_proc(proc);

    // 7. 返回新线程pid
    ret = proc->pid;
    return ret;

fork_out:
    return ret;
bad_fork_cleanup_kstack:
    put_kstack(proc);
bad_fork_cleanup_proc:
    kfree(proc);
    goto fork_out;
}

```

`do_fork()` 函数的整体思路为:按**分配PCB** → **建栈** → **复制/共享地址空间** → **设置内核上下文** → **分配PID并把子进程挂到进程表/哈希表** → **标记为可运行的顺序**创建一个新进程,并在每步失败时通过标签顺序释放已分配资源返回错误码。

以下是对每一步具体的分析:

1. 分配并初始化进程控制块

```

proc = alloc_proc();
if (!proc) {
    goto fork_out;
}

```

通过调用 `alloc_proc()` 分配并初始化 `proc_struct`,如果分配失败(内存不足或资源耗尽),函数直接退出并返回前面设定的 `-E_NO_MEM`。

2. 分配并初始化内核栈

```

if (setup_kstack(proc) != 0) {
    goto bad_fork_cleanup_proc;
}

```

调用 `setup_kstack()` 为新进程分配内核栈,并把 `proc->kstack` 等字段初始化,如果失败则跳转到 `bad_fork_cleanup_proc`,释放之前分配的`proc`(PCB),返回错误。

3. 复制或共享内存描述符

```
if (copy_mm(clone_flags, proc) != 0) {
    goto bad_fork_cleanup_kstack;
}
```

调用 `copy_mm()` 根据 `clone_flags` 决定子进程是否与父进程共享地址空间(`CLONE_VM`)或复制父进程的mm(写时复制),如果复制/分配页表、VMA等失败,需要回退:跳到 `bad_fork_cleanup_kstack`,释放内核栈和PCB。

4. 设置进程的中断帧和上下文

```
copy_thread(proc, stack, tf);
```

调用 `copy_thread()` 在子进程的内核栈顶部设置trapframe(拷贝父进程的tf并修改返回值、寄存器等),并初始化 `proc->context` (如返回地址、内核栈指针),使得当该进程被调度时会进入内核入口并正确返回到用户空间或内核函数。

5. 将子进程插入进程集合(链表与哈希表),并维护计数

```
proc->pid = get_pid();
list_add(&proc_list, &proc->list_link);
hash_proc(proc);
nr_process++;
```

这里首先调用 `get_pid()` 分配唯一PID,然后调用 `list_add()` 将子进程加入全局 `proc_list` (双向循环链表),调用 `hash_proc()` 将其加入PID哈希表或其他索引结构,并且通过 `nr_process++` 更新全局进程数。

6. 将子进程设为就绪态

```
wakeup_proc(proc);
```

调用 `wakeup_proc()` 将 `proc->state = PROC_RUNNABLE` 并把进程插入就绪队列,使调度器可选中该进程运行,从而把子进程从"初始化"变为系统可调度的实体。

7. 返回新进程PID

```
ret = proc->pid;
return ret;
```

成功时返回子进程的PID(正整数);失败路径会返回先前设置的负错误码。

`fork()` 函数通过以上步骤完成当前内核线程的一个副本的创建。

接下来回答练习2中提出的问题:

ucore能做到给每个新fork的线程一个唯一的id。原因如下:

在fork的实现中,对于我们上面实现的第五步部分内容如下:

```
proc->pid = get_pid();
```

可以看到,这里调用了 `get_pid()` 来给新进程分配PID,因此ucore能做到给每个新fork的线程一个唯一的id取决于 `get_pid()` 的实现逻辑,我们找到对应的 `get_pid()` 实现代码,如下所示:

```

static int
get_pid(void)
{
    static_assert(MAX_PID > MAX_PROCESS);
    struct proc_struct *proc;
    list_entry_t *list = &proc_list, *le;
    static int next_safe = MAX_PID, last_pid = MAX_PID;

    if (++last_pid >= MAX_PID)
    {
        last_pid = 1;
        goto inside;
    }

    if (last_pid >= next_safe)
    {
        inside:
        next_safe = MAX_PID;
        repeat:
        le = list;
        while ((le = list_next(le)) != list)
        {
            proc = le2proc(le, list_link);
            if (proc->pid == last_pid)
            {
                if (++last_pid >= next_safe)
                {
                    if (last_pid >= MAX_PID)
                    {
                        last_pid = 1;
                    }
                    next_safe = MAX_PID;
                    goto repeat;
                }
            }
            else if (proc->pid > last_pid && next_safe > proc->pid)
            {
                next_safe = proc->pid;
            }
        }
    }
    return last_pid;
}

```

`get_pid()` 的**整体思路**为:通过"循环PID + 跳跃优化"的策略,为新进程寻找一个当前未被占用的最小可用PID:先尝试使用 `last_pid` 的下一个值;如果该值被占用或不安全,则遍历进程链表收集所有已占用的PID,并借助 `next_safe` 快速跳过已知冲突区间,直到找到一个未被任何进程使用的PID为止。

其中关键的变量说明如下:

- `last_pid`: 记录"上次分配的PID",每次分配pid时,优先分配 `last_pid+1`,如果到达了最大则从1重来,循环递增尝试,这样能提升性能,使得分配的PID更均匀,不会一直从小PID开始,防止短时间内大量冲突。
- `next_safe`: 代表一个"安全上限"(即下一次可能冲突的最小PID),也就是表明从当前 `last_pid` 往上递增,在遇到 `next_safe` 前都不会和任何现存进程PID冲突,从而能够有效减少遍历链表的次数。

`get_pid()` 的**具体执行逻辑**如下:

1. 首先做一些准备工作:

```
static_assert(MAX_PID > MAX_PROCESS);
struct proc_struct *proc;
list_entry_t *list = &proc_list, *le;
static int next_safe = MAX_PID, last_pid = MAX_PID;
```

检查 `MAX_PID` 必须大于 `MAX_PROCESS`, 保证系统的可用PID数量比最大进程数多; 声明一个指向进程结构的指针, 用来拿到每个链表节点对应的进程结构; 定义两个链表指针变量: `list` 指向进程链表头 `proc_list`, 而 `le` 则作为遍历链表时使用的临时指针; 定义静态局部变量 `last_pid` 和 `next_safe`, 并把它们都初始化为 `MAX_PID`。

2. 然后 `last_pid` 自增1作为pid的候选值;

3. 接着判断 `last_pid` 是否大于等于 `MAX_PID` 和 `last_pid` 是否大于等于当前的安全上限 `next_safe`: 如果 `last_pid` 大于等于 `MAX_PID` (即超过了上限, 则从最大值绕回到起始值, 把 `last_pid` 设置为1) 或者 `last_pid` 大于等于当前的安全上限 `next_safe`, 都会进入扫描分支 (inside), 重新初始化 `next_safe` 并循环找到当前情况下的合适的 `last_pid` 和对应的 `next_safe`。

4. 扫描分支的具体处理为: 首先把 `next_safe` 重置为 `MAX_PID` (表示暂时不知道新的界限), 然后遍历 `proc_list` 中每个 `proc`:

①如果 `proc->pid == last_pid` (冲突):

```
if (proc->pid == last_pid)
{
    if (++last_pid >= next_safe)
    {
        if (last_pid >= MAX_PID)
        {
            last_pid = 1;
        }
        next_safe = MAX_PID;
        goto repeat;
    }
}
```

则进行 `++last_pid`, 尝试下一个PID, 并检查自增之后的 `last_pid` 是否大于等于 `next_safe`, 如果成立, 则说明 `last_pid` 达到或超过当前 `next_safe`, 需要重置 `next_safe` 并重新从链表头开始扫描 (`next_safe = MAX_PID; goto repeat;`), 而且在重置前如果 `last_pid >= MAX_PID`, `last_pid` 会被置为1; 否则即 `++last_pid < next_safe`, 则继续当前链表遍历 (不 `goto repeat`), 因为新的 `last_pid` 仍然在当前扫描所能判断的区间内, 遍历剩下的进程仍然可以确定是否冲突或更新 `next_safe`。

②否则如果 `proc->pid > last_pid` 并且 `proc->pid < next_safe`:

```
else if (proc->pid > last_pid && next_safe > proc->pid)
{
    next_safe = proc->pid;
}
```

则更新 `next_safe = proc->pid`。也就是记录“比 `last_pid` 大的最小已占用PID”, 用于后续判断。

最终当while遍历结束且没有再次触发 `goto repeat` (没有冲突导致重新开始), 说明在本次全表扫描里没有找到 `proc->pid == last_pid`, 此时 `last_pid` 是可用的, 函数跳出if-block并返回 `last_pid` (保证唯一)。

5. 如果没有进入这些if条件,则说明 `last_pid < next_safe`,由于上一次扫描已保证在 `[last_pid, next_safe)` 内没有占用的PID,因此能直接返回 `last_pid`,并且保证唯一。

综上执行流程,我们可以得出 `get_pid()` 得到的pid是唯一的,即ucore能做到给每个新fork的线程一个唯一的id。

练习3:编写proc_run函数(需要编码)

`proc_run` 用于将指定的进程切换到CPU上运行。它的大致执行步骤包括:

- 检查要切换的进程是否与当前正在运行的进程相同,如果相同则不需要切换。
- 禁用中断。你可以使用 `/kern/sync/sync.h` 中定义好的宏 `local_intr_save(x)` 和 `local_intr_restore(x)` 来实现关、开中断。
- 切换当前进程为要运行的进程。
- 切换页表,以便使用新进程的地址空间。 `/libs/riscv.h` 中提供了 `lsatp(unsigned int pgdir)` 函数,可实现修改SATP寄存器值的功能。
- 实现上下文切换。 `/kern/process` 中已经预先编写好了 `switch.S`,其中定义了 `switch_to()` 函数。可实现两个进程的context切换。
- 允许中断。

请回答如下问题:

在本实验的执行过程中,创建且运行了几个内核线程?

Ans: `proc_run` 函数是操作系统进程管理中的核心调度执行函数,它的作用是将指定的进程切换到CPU上运行。

根据实验要求,对 `proc_run` 函数进行补充,代码如下:

```
void proc_run(struct proc_struct *proc)
{
    if (proc != current)
    {
        bool intr_flag;
        struct proc_struct *prev = current;
        local_intr_save(intr_flag);
        {
            current = proc;
            lsatp(proc->pgdir);
            switch_to(&(prev->context), &(proc->context));
        }
        local_intr_restore(intr_flag);
    }
}
```

对 `proc_run` 函数进行分析:

```
void proc_run(struct proc_struct *proc)
```

函数的参数 `proc`,是指向要切换到的目标进程的进程控制块指针。

```
if (proc != current)
```

在函数体中,判断目标进程是否与当前进程相同:如果目标进程就是当前进程,说明进程已经在运行了。此时不需要切换,直接返回,避免不必要的开销。这是一种优化措施,可以提高系统效率。

```
bool intr_flag;  
struct proc_struct *prev = current;
```

如果目标进程与当前进程不同,则执行下列操作。

首先,声明中断标志变量 `intr_flag`,用于保存当前的中断状态。因为在后续需要关闭中断,所以在关闭前要记录中断原来的状态,方便恢复。

然后,用临时变量 `prev` 保存当前进程的指针。因为在下一步中会修改 `current` 指针,但 `switch_to` 函数需要用到旧进程的context地址。并且如果不保存,在修改 `current` 后就找不到旧进程了。

```
local_intr_save(intr_flag);
```

接下来,禁用中断,并将当前中断状态保存。进程切换是临界区操作,必须保证其原子性。

其实现原理为:通过清除 `sstatus` 寄存器的SIE位来禁用中断;并将原来的SIE位状态保存到变量 `intr_flag`。

```
{  
    current = proc;  
    lsatp(proc->pgdir);  
    switch_to(&(prev->context), &(proc->context));  
}
```

该代码块表示一个逻辑整体,在关闭中断状态下原子执行。

首先,更新当前进程指针,将全局变量 `current` 指向新进程。

然后,切换地址空间。修改SATP寄存器,切换到新进程的页表。

最后,进行上下文切换。保存旧进程的寄存器状态,恢复新进程的寄存器状态。将当前CPU的寄存器值保存到 `prev->context`,确保下次切换回来时能继续执行;从 `proc->context` 中加载寄存器的值到CPU;通过修改返回地址 `ra` 和栈指针 `sp` 的值,实现跳转。

当 `switch_to` 函数返回时,实际上是返回到新进程中。

```
local_intr_restore(intr_flag);
```

最后,恢复之前保存的中断状态。即根据 `intr_flag` 的值,恢复 `sstatus` 寄存器的SIE位。

这里有几个值得注意的点:

1. 要先更新current,再调用switch_to。

因为在 `switch_to` 返回后,代码实际上已经在新进程的上下文中执行了。如果先调用 `switch_to`,那么更新 `current` 的代码会在新进程中执行,这个顺序与正常逻辑不符。

2. 为什么switch_to返回后,还在proc_run函数中?

这是因为虽然进程切换了,但两个进程都是在 `proc_run` 中调用的 `switch_to` 函数。现在切换回来,从保存的返回地址继续执行,恰好就是 `switch_to` 返回的地址。通过对称性设计,让所有进程都通过同样的方式切换。

3. 如果在禁用中断期间发生硬件中断会怎样?

在禁用中断期间发生的硬件中断都不会立即响应,而是被挂起;当中断重新打开后,才会被处理。这种机制保证了进程切换不会被打断。

对练习三的问题进行回答(本实验的执行过程中,创建且运行了几个内核线程?)

创建并运行的内核线程共**2个**。

```
if ((idleproc = alloc_proc()) == NULL)
.....
idleproc->pid = 0;
idleproc->state = PROC_RUNNABLE;
idleproc->kstack = (uintptr_t)bootstack;
idleproc->need_resched = 1;
set_proc_name(idleproc, "idle");
nr_process++;
```

首先,在 `proc_init()` 中通过 `alloc_proc()` 手动创建线程 `idleproc`,然后对该线程进行初始化,命名为"idle"。该线程是一个空闲线程,当没有其他可运行线程时运行。

```
int pid = kernel_thread(init_main, "Hello world!!", 0);
if (pid <= 0)
{
    panic("create init_main failed.\n");
}
initproc = find_proc(pid);
set_proc_name(initproc, "init");
```

然后,在 `proc_init()` 中通过 `kernel_thread()` 创建第二个线程 `initproc`,命名为"init"。执行函数 `init_main()`,打印信息后退出。

执行顺序为:

```
kern_init()
└> proc_init() [创建线程的地方]
    ├──> 创建 idleproc (PID=0, 名称="idle")
    │   ├──> alloc_proc() 手动创建
    │   └> 设置为 PROC_RUNNABLE
    │
    └> 创建 initproc (PID=1, 名称="init")
        └> kernel_thread(init_main, "Hello world!!", 0)
            └> 通过 do_fork() 创建,设置为 PROC_RUNNABLE
                |
                └> cpu_idle() [idleproc 开始运行]
                    └> 不断检查 need_resched,调度器会选择运行 idleproc 或 initproc
```

最后,执行 `make` 和 `make qemu` 指令,编译代码并运行内核,实现效果如图所示。


```

(THU.CST) os is loading ...

Special kernel symbols:
  entry  0xc020004a (virtual)
  etext  0xc0203e96 (virtual)
  edata  0xc0209030 (virtual)
  end    0xc020d4ec (virtual)
Kernel executable memory footprint: 54KB
memory management: default_pmm_manager
physical memory map:
  memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
alloc_proc() correct!
++ setup timer interrupts
this initproc, pid = 1, name = "init"
To U: "Hello world!!".
To U: "en.., Bye, Bye. :)"
kernel panic at kern/process/proc.c:401:
  process exit!!.

Welcome to the kernel debug monitor!!
Type 'help' for a list of commands.
sunyy@sunyy-VMware-Virtual-Platform:~/桌面/lab4/lab4$

```

从运行结果可以看出,各部分初始化正常。也能够正常输出当前线程信息和"Hello world!!"字符串。最后,线程也可以正常退出。

```

gmake[1]: 进入目录"/home/sunyy/桌面/lab4/lab4" + cc kern/init/entry.S + cc kern/init/i
.c + cc kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/clock.c + cc ke
driver/picirq.c + cc kern/trap/trap.c + cc kern/trap/trapentry.S + cc kern/mm/best_fit
cc kern/mm/vmm.c + cc kern/process/entry.S + cc kern/process/proc.c + cc kern/process
cc libs/string.c + ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O
  -check alloc proc:                                OK
  -check initproc:                                  OK
Total Score: 30/30
sunyy@sunyy-VMware-Virtual-Platform:~/桌面/lab4/lab4$

```

执行 `make grade` 命令,查看程序得分为30/30。

扩展练习 Challenge

1. 说明语句 `local_intr_save(intr_flag);....local_intr_restore(intr_flag);` 是如何实现开关中断的?

为了回答这个问题,我们需要结合 `sync.h`、`riscv.h` 代码来看,这组语句的本质是先保存当前中断状态并禁用中断,执行临界区代码后,恢复到原来的中断状态,既保证临界区安全,又不破坏系统原有中断配置。

首先我们先来看 `kern/sync/sync.h`,这是核心封装。

该文件直接定义了 `local_intr_save/local_intr_restore` 宏,以及实际执行操作的辅助函数,我们在代码中加上一些注释来帮助理解:

```
// 引入 RISC-V 架构寄存器操作接口
#include <riscv.h>

// 辅助函数:保存当前中断状态,并禁用中断
static inline bool __intr_save(void) {
    // 1. 读取 RISC-V 的 sstatus 寄存器(特权级状态寄存器),判断 SIE 位(中断使能位)
    if (read_csr(sstatus) & SSTATUS_SIE) {
        intr_disable(); // 2. 若中断已开启,则禁用中断
        return 1;       // 3. 返回1,表示"原中断状态是开启"
    }
    return 0;          // 4. 返回0,表示"原中断状态是禁用"
}

// 辅助函数:根据保存的状态,恢复中断
static inline void __intr_restore(bool flag) {
    if (flag) {        // 若原状态是"开启"(flag=1)
        intr_enable(); // 重新开启中断
    }
}

// 对外暴露的宏:保存中断状态到 intr_flag,并禁用中断
#define local_intr_save(x) \
    do { \
        x = __intr_save(); \ // 调用__intr_save,结果存入 x(x 即 intr_flag)
    } while (0)

// 对外暴露的宏:根据 intr_flag 恢复中断状态
#define local_intr_restore(x) __intr_restore(x);
```

然后再结合看一下 `riscv.h` 的部分代码:

```
#define read_csr(reg) ({ unsigned long __tmp; \
    asm volatile ("csrr %0, " #reg : "=r"(__tmp)); \
    __tmp; })

#define set_csr(reg, bit) ({ unsigned long __tmp; \
    asm volatile ("csrrs %0, " #reg ", %1" : "=r"(__tmp) : "rk"(bit)); \
    __tmp; })

#define clear_csr(reg, bit) ({ unsigned long __tmp; \
    asm volatile ("csrrc %0, " #reg ", %1" : "=r"(__tmp) : "rk"(bit)); \
    __tmp; })
```

要理解代码,我们需要先明确RISC-V架构的两个关键规则:

1. 中断使能由sstatus寄存器的SIE位控制:

- `SIE=1`:内核态(Supervisor态)中断开启,CPU能响应外部中断(如定时器中断、I/O中断);
- `SIE=0`:内核态中断禁用,CPU忽略所有外部中断。

2. 寄存器操作必须通过特权指令:

- 读取 `sstatus` 用 `csrr` 指令,写入用 `csrw` 指令。

了解完以上的前提信息后,我们可以来拆解这段语句的完整执行流程,从保存到恢复:

以 `local_intr_save(intr_flag);` 临界区代码; `local_intr_restore(intr_flag);` 为例,一步步看如何实现开关中断:

步骤1:local_intr_save(intr_flag)—— 保存状态 + 禁用中断

宏展开后执行 `intr_flag = __intr_save();`, `__intr_save()` 做三件事:

1. 判断当前中断状态:

执行 `read_csr(sstatus) & SSTATUS_SIE`:读取 `sstatus` 寄存器,用与运算提取SIE位的值(0或1)。

- 若结果为1:说明当前中断是"开启"的;
- 若结果为0:说明当前中断已"禁用"。

2. 强制禁用中断:

- 若原状态为开启(SIE=1),则调用 `intr_disable()` 清除SIE位;
- 若原状态已禁用(SIE=0),则无需操作,直接返回。

3. 保存原状态到intr_flag:

返回原中断状态(1 = 开启,0 = 禁用),存入变量 `intr_flag` (相当于"记下之前的开关状态")。

步骤2:执行临界区代码

此时 `SIE=0`,内核态中断已禁用,CPU不会响应任何外部中断,这确保了临界区代码(如进程上下文切换、修改全局链表 `proc_list`)能完整执行,不会被打断导致数据不一致。

步骤3:local_intr_restore(intr_flag)—— 恢复原中断状态

调用 `__intr_restore(intr_flag)`,根据之前保存的 `intr_flag` 恢复:

- 若 `intr_flag=1`:说明进入临界区前中断是开启的,执行 `intr_enable()` 设置SIE位(置1),重新开启中断;
- 若 `intr_flag=0`:说明进入临界区前中断就已禁用,不做任何操作。

这样,通过 `local_intr_save/restore` 的设计,只在临界区临时禁用中断,退出后恢复到原来的状态,不会干扰系统其他逻辑,这是操作系统临界区保护的标准设计,保证了代码的通用性和安全性。

2. 深入理解不同分页模式的工作原理(思考题)

`get_pte()` 函数(位于 `kern/mm/pmm.c`)用于在页表中查找或创建页表项,从而实现对指定线性地址对应的物理页的访问和映射操作。这在操作系统中的分页机制下,是实现虚拟内存与物理内存之间映射关系非常重要的内容。

`get_pte()` 函数中有两段形式类似的代码,结合sv32,sv39,sv48的异同,解释这两段代码为什么如此相像。

目前 `get_pte()` 函数将页表项的查找和页表项的分配合并在一个函数里,你认为这种写法好吗?有没有必要把两个功能拆开?

Ans: 首先我们先来看一下 `get_pte()` 函数中的两段相似代码:

```
// 第一段:处理第一级页目录 (PDX1)
pde_t *pdep1 = &pgdir[PDX1(la)];
if (!(*pdep1 & PTE_V)) {
```



```

    struct Page *page;
    if (!create || (page = alloc_page()) == NULL) {
        return NULL;
    }
    set_page_ref(page, 1);
    uintptr_t pa = page2pa(page);
    memset(KADDR(pa), 0, PGSIZE);
    *pdep1 = pte_create(page2ppn(page), PTE_U | PTE_V);
}

// 第二段:处理第二级目录 (PDX0)
pde_t *pdep0 = &((pte_t *)KADDR(PDE_ADDR(*pdep1)))[PDX0(1a)];
if (!(*pdep0 & PTE_V)) {
    struct Page *page;
    if (!create || (page = alloc_page()) == NULL) {
        return NULL;
    }
    set_page_ref(page, 1);
    uintptr_t pa = page2pa(page);
    memset(KADDR(pa), 0, PGSIZE);
    *pdep0 = pte_create(page2ppn(page), PTE_U | PTE_V);
}

```

来解释一下为什么会如此相似:

首先从分页模式的层级共性来看,RISC-V的SV32(2级页表)、SV39(3级页表)、SV48(4级页表)均采用多级页表结构,每级页表的功能和结构一致:

- 每级页表项(PTE)存储下一级页表的地址或最终物理页号。
- 各级页表的大小均为1页(如4KB),页表项数量由页大小和页表项宽度决定(如SV39中每级有512个页表项)。

然后我们来考虑代码复用性,两级代码分别对应多级页表中的连续两级索引,操作逻辑完全相同:

- 计算当前级页表的索引(PDX1对应一级索引,PDX0对应二级索引)。
- 检查页表项是否有效(PTE_V标志)。
- 若无效且需要创建(create=1),则分配新页作为下一级页表,并初始化页表项。

最后是扩展性适配,若未来支持更多级页表(如SV48的4级),只需按相同逻辑增加对应层级的代码,保持结构一致性。

所以这其实是硬件设计的规律性,也恰恰体现了RISC-V的设计哲学。

接下来我们回答一下第二个问题,功能合并与拆分的分析。

当前 `get_pte()` 函数同时实现了页表项查找和页表分配合并的功能(通过 `create` 参数控制是否分配新页表)。那么这种设计一定是存在合理性的,我们可以来分析具体的优劣势再决定是否拆分:

首先是优点,可以概括如下:

1. **原子性**:查找和分配在同一函数中完成,避免了多线程环境下的竞态条件。
2. **简洁性**:对于分页机制的核心操作,无需单独调用查找和分配函数,减少代码冗余。
3. **效率**:一次函数调用完成两级页表的索引,减少函数调用开销。

但是也存在着很明显的缺点:

1. **职责不单一**:一个函数处理两种逻辑,可读性和可维护性下降。
2. **灵活性不足**:若仅需查询页表项(create=0),仍需执行与分配相关的条件判断代码。
3. **测试难度增加**:函数逻辑较复杂,单元测试需覆盖"查找成功""查找失败""分配成功""分配失败"等多种场景。

如果仅从当前的实验环境下来看,目前的写法是可以的,而且可以正常工作,但若在实际的生产系统中,考虑拆分可能更好,原因如下:

- **模块化设计:**拆分后, `find_pte()` 负责纯查询, `create_pte()` 负责分配新页表,职责清晰。
- **可复用性:**某些场景下只需查询页表(如 `get_page`),可单独调用 `find_pte()` 。
- **可维护性:**拆分后代码逻辑更简单,便于调试和扩展,如支持更多级页表时只需修改 `create_pte()` 。

总而言之,功能合并与拆分是在接口简洁性和功能灵活性之间的合理权衡,应考虑实际应用场景来看。

最后,附上一些SV32, SV39, SV48的异同对比:

特性	SV32	SV39	SV48
虚拟地址位宽	32位	39位	48位
物理地址位宽	34位	56位	56位
页表级数	2级	3级	4级
每级索引位宽	10位	9位	9位
页大小	4KB	4KB	4KB

四、讨论

实验知识点与OS原理知识点对应关系

实验中的重要知识点	对应的OS原理知识点	含义、关系及差异理解
进程控制块(PCB,struct proc_struct)的分配与初始化 (alloc_proc函数)	进程控制块 (PCB) 机制	含义:实验中PCB是存储内核线程管理信息的核心数据结构,包含状态、PID、内核栈、上下文等成员;OS原理中PCB是操作系统描述和管理进程的关键数据结构,是进程存在的唯一标识。 关系:实验中的PCB实现完全遵循OS原理中PCB的核心设计思想,是原理的工程化落地。 差异:实验中PCB针对内核线程简化设计(如mm=NULL),原理中PCB需适配用户进程、内核线程等多种执行实体,成员更复杂(如资源描述符、优先级等)。

实验中的重要知识点	对应的OS原理知识点	含义、关系及差异理解
内核线程创建流程(do_fork函数:分配PCB、内核栈、复制上下文、加入进程链表)	进程创建机制(fork系统调用原理)	含义:实验中通过do_fork复制现有内核线程资源创建新线程;OS原理中进程创建是通过父进程复制资源(如地址空间、PCB)生成子进程,是多进程并发的基础。 关系:实验中的do_fork是OS原理中fork机制的简化实现,核心逻辑(资源复制、状态初始化、加入调度队列)完全一致。 差异:实验仅支持内核线程创建,无需处理用户地址空间复制;原理中fork需支持写时复制(COW)、地址空间隔离等复杂逻辑。
上下文切换(switch_to函数:保存/恢复struct context寄存器)	进程上下文切换原理	含义:实验中通过汇编函数保存当前线程的被调用者保存寄存器,恢复目标线程寄存器;OS原理中上下文切换是保存当前进程执行现场,恢复目标进程现场,实现CPU使用权转移的核心操作。 关系:实验中的上下文切换是原理的底层实现,严格遵循"保存-恢复"的核心流程。 差异:实验仅保存被调用者保存寄存器(编译器自动处理调用者保存寄存器),原理中需根据架构差异确定保存的寄存器集合,逻辑更通用。
进程调度(FIFO调度策略,schedule函数查找就绪进程)	进程调度算法(批处理系统调度策略)	含义:实验中采用简单FIFO调度,从proc_list中查找下一个PROC_RUNNABLE状态的线程;OS原理中调度算法是操作系统按一定规则分配CPU资源的策略,FIFO是最基础的非抢占式调度算法。 关系:实验中的FIFO调度是原理中基础调度算法的直接实现,用于验证调度机制的可行性。 差异:实验仅实现单一FIFO策略,原理中调度算法需考虑公平性、高效性(如时间片轮转、优先级调度),支持抢占式调度。
中断控制(local_intr_save/local_intr_restore宏开关中断)	临界区保护机制(中断屏蔽)	含义:实验中通过操作sstatus寄存器的SIE位禁用/恢复中断,保护进程切换等临界区操作;OS原理中中断屏蔽是临界区保护的重要手段,通过禁止中断避免并发操作导致的数据不一致。 关系:实验中的中断控制是原理中中断屏蔽机制的硬件级实现,完全遵循"进入临界区关中断,退出临界区开中断"的原则。 差异:实验仅依赖RISC-V架构的sstatus寄存器操作,原理中中断屏蔽需适配不同架构(如x86的IF标志位),且需结合信号量、互斥锁等其他临界区保护手段。

实验中的重要知识点	对应的OS原理知识点	含义、关系及差异理解
多级页表查找与页表项管理 (get_pte函数:SV39三级页表索引、页表项创建)	多级页表机制	含义:实验中get_pte通过PDX1、PDX0、PTX三级索引查找页表项,按需创建各级页表;OS原理中多级页表是解决单级页表内存浪费问题的技术,将页表分为多个层级,按需分配页表空间。 关系:实验中的SV39多级页表是原理中多级页表机制的具体实现,完全遵循"地址拆分-层级索引-按需分配"的核心逻辑。 差异:实验固定使用三级页表(SV39),原理中多级页表支持不同层级(如二级、四级),需适配不同虚拟地址宽度需求。
虚拟地址到物理地址映射 (page_insert/page_remove函数)	虚拟内存映射原理	含义:实验中通过页表项建立虚拟地址与物理页的对应关系,实现地址转换;OS原理中虚拟内存映射是将进程虚拟地址空间映射到物理内存,实现地址隔离、内存复用的基础。 关系:实验中的映射操作是原理的工程实现,严格遵循"页表项记录映射关系,CPU通过页表完成地址转换"的逻辑。 差异:实验采用预映射方式,无缺页异常处理;原理中虚拟内存映射需结合按需分页、缺页异常、页面置换等机制。

OS原理中重要但实验未覆盖的知识点

- 进程同步与互斥机制:**OS原理中进程同步(如信号量、管程)和互斥是解决多进程并发访问共享资源冲突的核心技术,是多进程协作的基础。实验中仅实现简单多线程调度,未涉及共享资源竞争场景,因此未实现相关机制。
- 进程通信机制:**OS原理中进程通信(如管道、消息队列、共享内存)是实现进程间数据交换和协作的关键技术。实验中内核线程无进程间通信需求,仅需独立执行任务,因此未覆盖该知识点。
- 抢占式调度与优先级调度算法:**OS原理中抢占式调度(基于时间片或优先级抢占CPU)和优先级调度是保证实时性、公平性的重要调度策略,广泛应用于实际操作系统。实验中仅实现非抢占式FIFO调度,未支持抢占机制和优先级配置。
- 用户进程管理与用户态-内核态切换:**OS原理中用户进程是操作系统的主要执行实体,用户态-内核态切换(通过系统调用、中断)是隔离用户程序与内核、保护系统安全的核心机制。实验仅支持内核线程,未涉及用户进程的地址空间构建、系统调用接口等内容。
- 内存置换算法与按需分页:**OS原理中按需分页(仅在访问虚拟地址时分配物理内存)和内存置换算法(如LRU、Clock)是解决物理内存不足、提高内存利用率的关键技术。实验中采用预映射方式分配内存,无缺页异常处理和页面置换逻辑。
- 进程终止与资源回收机制:**OS原理中进程终止需回收PCB、地址空间、打开文件等所有资源,处理僵尸进程、孤儿进程等场景。实验中未实现完整的进程终止逻辑(do_exit函数仅抛出恐慌),未涉及资源回收的复杂处理。

五、实验结论及心得体会

实验结论

在本次实验中成功完成了uCore OS进程管理的核心功能,包括进程控制块的分配与初始化、内核线程的创建资源分配、进程切换执行函数的编写,所有核心练习均通过测试,最终得分为30/30。

实现了从单一执行流到多线程并发的内核扩展,搭建了基础虚拟内存环境,创建并运行了 `idleproc` (空闲线程)和 `initproc` (初始化线程)两个内核线程,实现了线程的调度与上下文切换。

验证了关键机制的有效性: `get_pid` 函数能为新fork线程分配唯一PID, `local_intr_save` 与 `local_intr_restore` 宏可安全控制中断,保障进程切换的原子性,页表切换与上下文恢复协同工作确保了地址空间的正确切换。

心得体会

通过本次实验,我们对操作系统进程管理的底层逻辑有了具象化认知,从PCB初始化、资源分配到进程切换,每一步都需严格遵循原子性和规范性,任何成员未初始化或顺序错误都可能导致内核崩溃,深刻体会到操作系统设计的严谨性。

通过亲自搭建PCB、实现fork逻辑、编写context切换代码,让我们深刻体会到:PCB是"进程的灵魂",必须保持完整性与一致性;内核线程并不是一个简单的函数,而是伴随独立资源(内核栈、上下文、中断帧)的完整执行实体;调度机制必须同时管理多种资源状态,是一个高度系统化的过程。

通过分析 `local_intr_save` 和 `local_intr_restore`,加深了对中断控制在进程切换中意义的理解:中断屏蔽不是"禁止硬件中断",而是通过SIE位让内核暂时忽略中断,这对保证调度的原子性至关重要。

本次实验不仅成功实现了uCore进程管理的核心功能,更帮助我们工程与理论的层面深入理解了操作系统内核的结构与运行机制。通过本次实验,真正体会到从"代码层面"构建一个操作系统所需要的严谨性、系统性与细致性。